Clase 3 — Encapsulación y comparación OSI ↔ TCP/IP

Bloque: A (Fundamentos) · **Semana:** 1 · **Duración estimada:** 110 min **Correlativa:** Clase 2 (los modelos de capas: OSI y TCP/IP) · **Sigue con:** Clase 4 (conmutación: circuitos, paquetes y mensajes) **Material de referencia:** Tanenbaum cap. 1 §1.4 (pág. ~20 a ~26 del PDF de tu carpeta) · RFC 8200, RFC 9293, RFC 768, RFC 826, RFC 1122

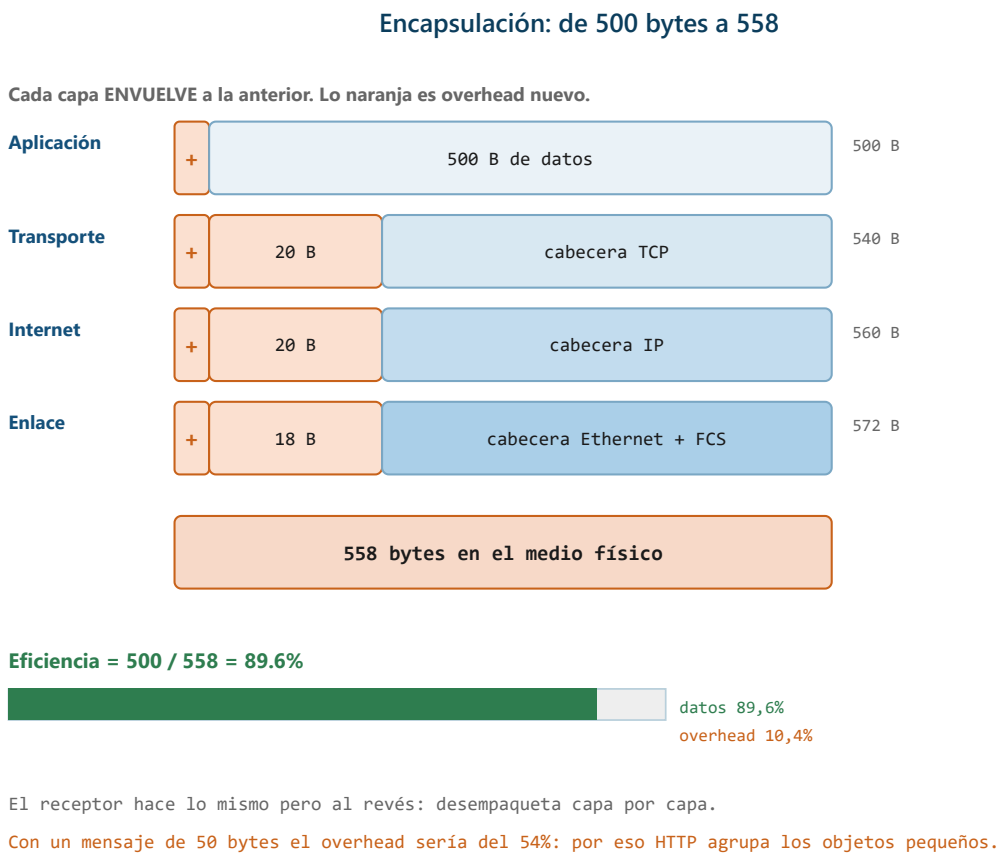
🎯 Al terminar esta clase vas a poder

1. **Definir** encapsulación y desencapsulación, y explicar **por qué existen** (cada capa necesita su propio control).
2. **Hacer la cuenta byte a byte** de un mensaje que baja por la pila: 500 B de aplicación → 558 B en el medio, con la aritmética visible.
3. **Calcular la eficiencia** en función del tamaño del mensaje y explicar **por qué el overhead relativo se dispara con mensajes chicos**.
4. **Dibujar** el camino completo de un mensaje con los dos extremos, y **marcar dónde se invierte** la encapsulación.
5. **Explicar** qué más cambia al bajar de capa además del tamaño: **el direccionamiento** (IP → MAC) y **el objetivo** (llegar al host → llegar al proceso).
6. **Describir** qué es un **servicio** entre capas y por qué es la clave de la encapsulación.
7. **Desarrollar** la capa de Presentación con casos concretos, porque es la que más se olvida.
8. **Resolver entero** el Examen `2009-02 P2` (forwarding vs. routing y flooding), que es la demostración de que la capa 3 existe por el alcance.

> **Prerequisito del examen:** `2009-02 P2` (10 pts — forwarding vs. routing, flooding controlado), `2011-03 Problema 2` (el intercambio de tramas con ARP), `2006-12 P1` y `2007-03 P1` (modelos de capas), `2009-12 P1` (componentes del retardo).

PARTE 1 — TEORÍA

1.1 Qué es la encapsulación y por qué existe



La definición

Encapsular es el proceso por el cual, al bajar por la pila, cada capa **antepone su propia cabecera** (y a veces su trailer) a los datos que le entrega la capa de arriba. **Desencapsular** es lo inverso: al subir, cada capa **quita su propia cabecera** y le entrega a la de arriba la PDU que tenía abajo.

```
                ENCAPSULACIÓN (bajar)
Aplicación  ──────────────────────────────►
Transporte  ──────────────────────────────►   + 20 B
Red         ──────────────────────────────►   + 20 B
Enlace      ──────────────────────────────►   + 18 B
Física      ──────────────────────────────►   bits


                DESENCAPSULACIÓN (subir)
Física      ◄──────────────────────────────
Enlace      ◄──────────────────────────────   − 18 B
Red         ◄──────────────────────────────   − 20 B
Transporte  ◄──────────────────────────────   − 20 B
Aplicación  ◄──────────────────────────────
```

"Encapsular" es "meter algo dentro de una cápsula". La cápsula es la cabecera. Lo que viaja por el medio es la cápsula con el contenido adentro.

Las cuatro razones por las que existe

Razón 1 — Control local. Si cada capa metiera sus cosas **al principio** de la unidad de datos, entonces una capa de arriba no podría volver a leer el mensaje sin recorrer todo. Con la cabecera **delante**, cada capa puede **encontrar y quitar la suya en tiempo constante**: la de transporte está en el byte 0, la de red en el byte 20 (si el offset de datos de transporte es 0) o en el byte 34, y la de enlace en el byte 14 del cable. Es el mismo motivo por el que el **TCP payload offset** existe: para que la capa IP sepa dónde empieza su propia cabecera sin recorrer la trama entera.

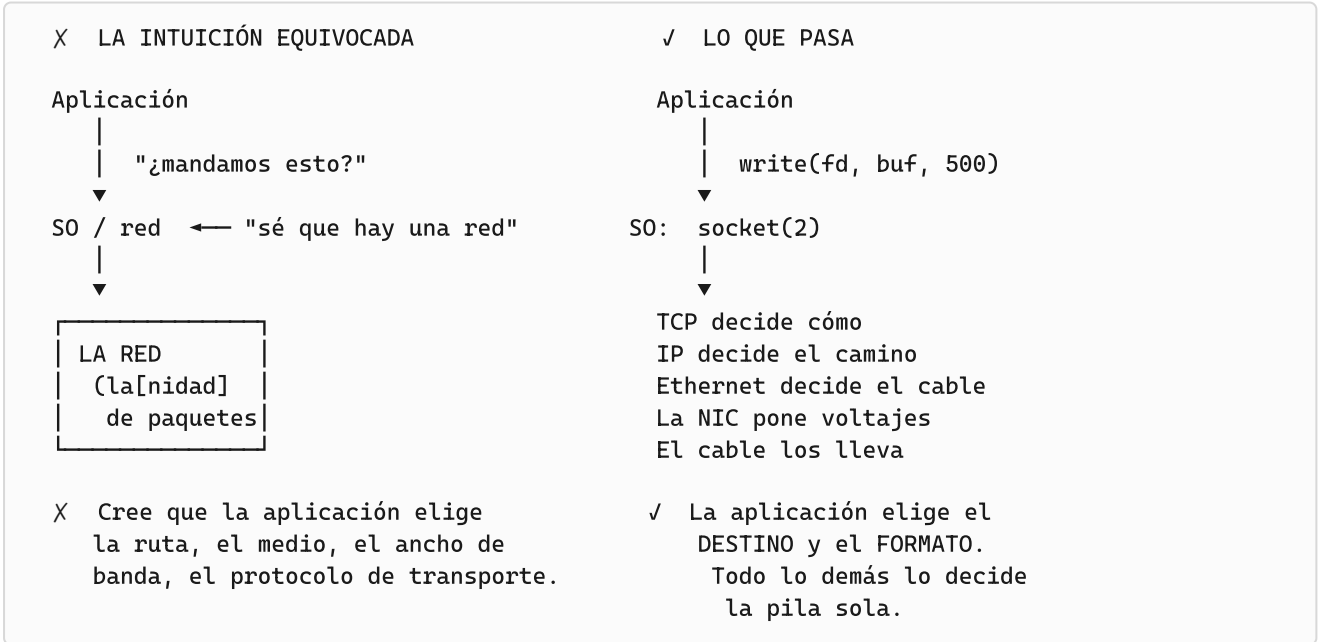
Razón 2 — Multiplexación. Con las cabeceras al principio, un router puede leer la dirección de destino **sin leer el payload**. Mira los 20 bytes de cabecera IP, decide el puerto de salida, y reenvía: **no tiene que abrir el sobre**. Si la dirección estuviera al final, el router tendría que recorrer los 1500 bytes para enterarse adónde va. Es la razón de performance más importante.

Razón 3 — cada capa habla su propio idioma. La capa de enlace necesita decir "esto es una trama Ethernet con destino esta MAC"; la capa de red necesita decir "esto va a 10.0.0.7 con TTL 63". Son **afirmaciones que ninguna otra capa puede hacer ni verificar**. La cabecera es el lugar donde cada capa deja su información.

Razón 4 — Independencia. Si el medio pasa de cobre a fibra, **solo cambia la capa 1**; los bytes de las capas 2 a 7 no se tocan. Si IP pasa de v4 a v6, **solo cambia la capa 3**; TCP no se entera. La encapsulación es lo que hace posible el **intercambio de una capa sin arrastrar a las demás**.

El error conceptual más frecuente: la aplicación "ve" la red

Hay una intuición muy extendida y **equivocada**: que la capa de aplicación "sabe" que está usando una red, o que "el paquete viaja por la red como una entidad visible".



Por qué importa en el examen. Si creés que la aplicación elige el protocolo de transporte, no entendés por qué **HTTP sobre TCP y HTTP sobre QUIC usan la misma aplicación** (el navegador), ni por qué **un `ping` no usa ninguna aplicación**. Si creés que la aplicación "ve" los paquetes, no entendés por qué **un sniffing a nivel de socket no ve nada de IP**.

La frase exacta: *la capa de aplicación nunca ve la red. Le entrega bytes a la capa de transporte mediante una interfaz, y recibe bytes. La red existe para ella como un detalle de implementación de la máquina donde corre.*

1.2 El diagrama completo: un byte viajando

Los datos

```
Aplicación en H1 escribe una solicitud HTTP de 500 bytes.
L_app = 500 bytes

Cabeceras del escenario:
  cabecera TCP           = 20 bytes  (sin opciones)
  cabecera IPv4          = 20 bytes  (sin opciones)
  trama Ethernet         = 18 bytes  (14 de cabecera + 4 de FCS)
  preámbulo + IFG       =  8 bytes  (7 preámbulo + 1 SFD, + 12 de IFG
                                      que van entre tramas, NO dentro)
```

> ⚠️ **Precisión que importa:** el **preámbulo (7 bytes) y el SFD (1 byte)** son parte de la trama en el medio, pero **no forman parte de la PDU de capa 2** que se le entrega a la capa 3. Y el **IFG (Inter Frame Gap) de 96 bits = 12 bytes** va **entre** tramas, no dentro. Cuando un problema de examen dice "tamaño de la trama Ethernet", casi siempre quiere decir **14 + payload + 4**, sin preámbulo. Lo aclaro abajo.

La cuenta, paso a paso

Paso 1 — Capa 4: el segmento TCP.

```
Segmento TCP = cabecera TCP + datos de aplicación
             = 20 B        + 500 B
             = 520 bytes
```

Paso 2 — Capa 3: el paquete IP.

```
Paquete IP = cabecera IP + segmento TCP
           = 20 B        + 520 B
           = 540 bytes
```

Paso 3 — Capa 2: la trama Ethernet.

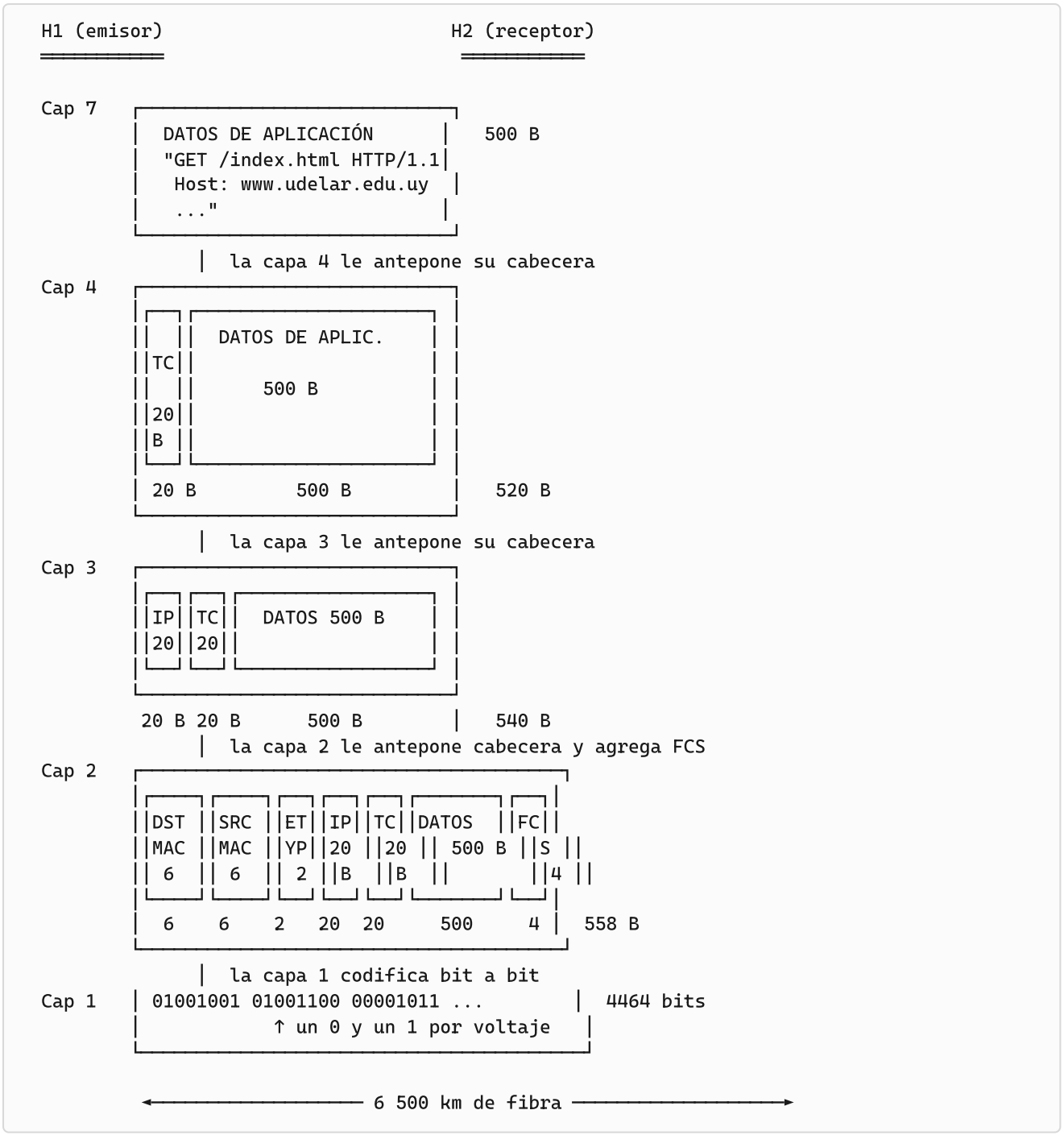
```
Trama = MAC dest (6) + MAC orig (6) + Ethertype (2) + datos + FCS (4)
      = 6 + 6 + 2 + 540 + 4
      = 558 bytes
```

Paso 4 — El total y el overhead.

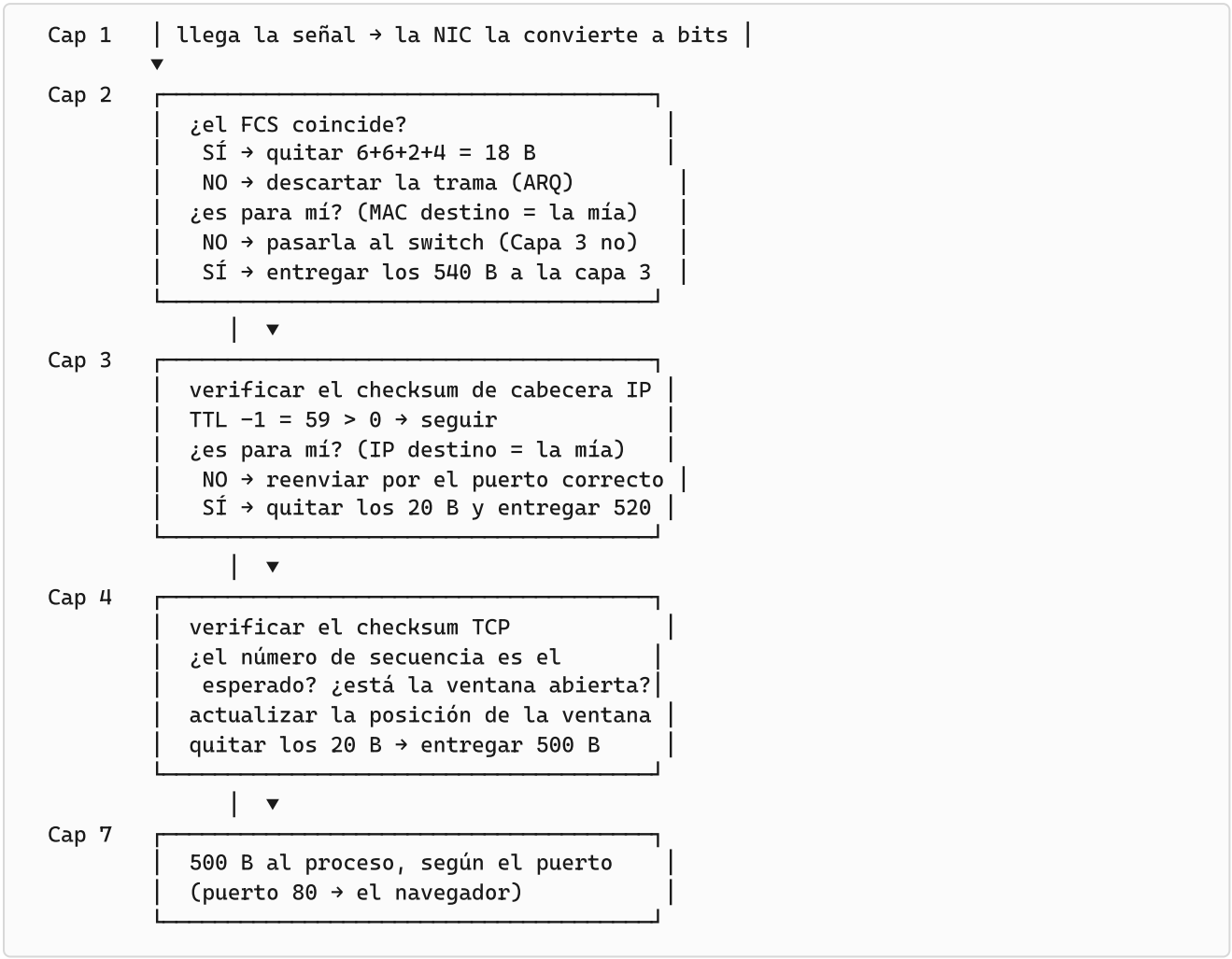
```
Bytes de datos útiles           = 500
Bytes de overhead              = 20 + 20 + 18 = 58
──────────────────────────────────────────────────
Bytes totales en el medio       = 558

OVERHEAD ABSOLUTO  = 58 bytes
OVERHEAD RELATIVO  = 58 / 558 = 10,39 %
EFICIENCIA         = 500 / 558 = 89,61 %
```

Paso 5 — El diagrama visual del byte viajando.


H1 (emisor)
H2 (receptor)
Cap 7
DATOS DE APLICACIÓN
"GET /index.html HTTP/1.1
Host: www.udelar.edu.uy
..."
500 B
la capa 4 le antepone su cabecera
Cap 4
TC
20
B
DATOS DE APLIC.
500 B
20 B
500 B
520 B
la capa 3 le antepone su cabecera
Cap 3
IP
20
TC
20
DATOS 500 B
20 B 20 B
500 B
540 B
la capa 2 le antepone cabecera y agrega FCS
Cap 2
DST MAC
SRC MAC
ET YP
IP
TC
DATOS
FC
6
6
2
20
20
500 B
4
6
6
2
20
20
500
4
558 B
la capa 1 codifica bit a bit
Cap 1
01001001 01001100 00001011 ...
4464 bits
↑ un 0 y un 1 por voltaje
6 500 km de fibra

La desencapsulación, del otro lado



Lo que hay que ver en este diagrama: la curva es simétrica. **Cada capa quita exactamente los bytes que antepuso.** Los 500 bytes que escribió la aplicación de H1 son **bit a bit** los mismos 500 bytes que lee la aplicación de H2. Lo único que hace el camino es **agregar 58 bytes de andamiaje** y sacarlos.

La tabla de tamaños

Capa	PDU	cabecera	datos	total
7 Aplicación	mensaje	0 B	500 B	500 B
4 Transporte	segmento	20 B	500 B	520 B
3 Red	paquete	20 B	520 B	540 B
2 Enlace	trama	18 B	540 B	558 B
1 Física	bits	−	540 B	4464 bits

overhead acumulado: +20 +40 +58

1.3 El overhead: la cuenta que siempre preguntan

La fórmula

```
Para un mensaje de L bytes de datos con cabeceras de tamaño fijo:

                  ┌──────────────────────────────────────┐
                  │                                      │
                  │        L                             │
eficiencia  =     ──────────────────────  =                 │
                  │        L + overhead                  │
                  └──────────────────────────────────────┘

overhead = H_TCP + H_IP + H_ENLACE
         =    20   +  20  +    18      = 58 bytes
```

El punto clave que hay que entender: el overhead es **constante** (58 bytes) pero el mensaje **no**. Entonces:

- Para mensajes **grandes**, el 58 es una fracción chica → eficiencia alta.
- Para mensajes **chicos**, el 58 es una fracción enorme → eficiencia bajísima.

La cuenta con tres tamaños (este es el tipo de pregunta que aparece)

```
overhead = 58 bytes (20 TCP + 20 IP + 18 Ethernet)

┌──────────┬──────────┬──────────┬──────────┬──────────────┐
│ L_app    │ total    │ eficiencia│ overhead %│ comentario   │
├──────────┼──────────┼──────────┼──────────┼──────────────┤
│   50 B   │  50 + 58 │ 50/108   │          │              │
│          │  = 108 B │ = 46,3 % │   53,7 % │ ¡peor que 50%!│
├──────────┼──────────┼──────────┼──────────┼──────────────┤
│  100 B   │ 100 + 58 │ 100/158  │          │              │
│          │  = 158 B │ = 63,3 % │   36,7 % │              │
├──────────┼──────────┼──────────┼──────────┼──────────────┤
│  500 B   │ 500 + 58 │ 500/558  │          │              │
│          │  = 558 B │ = 89,6 % │   10,4 % │              │
├──────────┼──────────┼──────────┼──────────┼──────────────┤
│ 1500 B   │1500 + 58 │1500/1558 │          │              │
│          │ = 1558 B │ = 96,3 % │    3,7 % │ MTU de Eth.  │
├──────────┼──────────┼──────────┼──────────┼──────────────┤
│ 64 000 B │64000 + 58 │64000/…  │          │              │
│          │=64058 B  │ = 99,91 % │   0,09 % │              │
└──────────┴──────────┴──────────┴──────────┴──────────────┘
```

El caso extremo: mensaje de 50 bytes

Este es el que **siempre se pide**, y el número es mengingatkan:

```
L_app = 50 bytes
overhead = 58 bytes
─────────────────────────────
total = 108 bytes

eficiencia = 50/108 = 0,4629 = 46,3 %
overhead   = 58/108 = 0,5370 = 53,7 %   ← MÁS DE LA MITAD

El mensaje de aplicación ocupa menos de la mitad de lo que
viaja por el cable. La red está transportando overhead.
```

El caso de 40 bytes (que es el mínimo absoluto):

```
L_app = 40 B
total = 40 + 58 = 98 B
eficiencia = 40/98 = 40,8 %
overhead   = 58/98 = 59,2 %
```

Y el caso de 1 byte:

```
L_app = 1 B
total = 1 + 58 = 59 B
eficiencia = 1/59 = 1,7 %
→ 98,3 % del tráfico es overhead. Absurdo.
```

Por qué esto importa de verdad (no es un ejercicio abstracto)

Motivo 1 — El MTU. El MTU de Ethernet son **1500 bytes de payload**. El tamaño máximo de un datagrama IP que cabe es `1500 − 20 (IP) − 20 (TCP) = 1460 bytes` de datos de aplicación, que es el **MSS (*Maximum Segment Size*) de TCP**. Los 40 bytes restantes los pone Ethernet. Es exactamente la misma cuenta, vista al revés.

Motivo 2 — La Ethernet no permite frames de menos de 64 bytes. Si tu mensaje de aplicación mide 10 bytes, el total sería 68 bytes, que ya supera el mínimo... pero el mínimo existe por otra razón: **el emisor tiene que transmitir al menos 64 bytes para que el receptor pueda tener tiempo de detectar una colisión**. Por eso las tramas se rellenan con *padding* hasta 64 bytes.

Motivo 3 — Protocolos que optimizan. Cuando importan los mensajes chicos:

- **UDP** tiene **8 bytes** de cabecera, no 20. Un datagrama de voz con 20 bytes de audio tiene 8/28 = 28,6 % de overhead en vez de 53,7 %. **Por eso el códec del RTP mete 20 bytes de audio por paquete y no 100**: cada ms de audio a 20 bytes es un paquete, y el tamaño de cabecera importa.
- **HTTP/2 (RFC 9113)** comprime las cabeceras con **HPACK (RFC 7541)**: una Headers repetitiva de 80 bytes puede quedar en 20.
- **QUIC** (RFC 9000) usa **cifrado integrado**, lo que agrega entre 16 y 40 bytes al inicio pero **elimina el handshake de 3 vías**, y el resultado es menos round trips aunque el sobre sea mayor.

Motivo 4 — La MTU y la fragmentación. Un datagrama IP de 2000 bytes **no cabe** en el MTU de 1500. O se fragmenta, o se descarta con ICMP. Cada fragmento lleva sus **propios 20 bytes de cabecera IP**. Fragmentar un mensaje de 2000 en 2 fragments agrega 20 bytes de overhead extra. Es la capa 3 reproduciendo el mismo efecto.

1.4 Lo que además cambia al bajar de capa

El tamaño no es lo único que cambia. Hay **dos cambios más**, y son los que de verdad justifican la encapsulación.

(a) El direccionamiento: de IP a MAC

```
ENCAPSULANDO (H1 → la red):

  El paquete IP dice:  192.168.1.25  →  8.8.8.8     (CAPA 3)
  La trama dice:       AA:BB:CC:DD:EE:01 → 00:1A:2B:3C:4D:5E   (CAPA 2)

  ¿Por qué NO puede ser la misma dirección en las dos capas?

  Porque son cosas distintas:
  ──────────────────────────────────────────────────────────────
  IP  = IDENTIDAD GLOBAL del host, JERÁRQUICA y ESTABLE.
        El paquete IP viaja de extremo a extremo: la IP destino
        no cambia en 12 routers. Es la que sabe el servidor de
        DNS, la que escribe la ACL del firewall, la que aparece
        en los logs del servidor.

  MAC = IDENTIDAD LOCAL del ENLACE, PLANA y REESCRIBIBLE.
        Solo tiene sentido dentro de un mismo medio físico.
        Cambia en CADA router:
           enlace 1:  AA:BB:CC:DD:EE:01 → 00:1A:2B:3C:4D:5E
           enlace 2:  FF:11:22:33:44:07 → 00:AA:BB:CC:DD:99
           enlace 3:  ...
  ──────────────────────────────────────────────────────────────

DESENCAPSULANDO (una trama llega a un router):

  El router hace:  NIC recibe la trama, ve que MAC destino
                   es la suya → se la pasa a la capa 3.
  La capa 3 hace:  lee la IP destino 8.8.8.8.
                   "No es mío." → busca en la tabla de reenvío.
                   "Va por eth1."  → se lo dice a la capa 2.
  La capa 2 hace:  busca en la tabla ARP qué MAC tiene eth1,
                   arma una trama nueva, la manda.

  → El router NUNCA ve la IP origen ni la IP destino en la trama.
    Las ve en el PAQUETE, que es otra cosa.
```

El corolario que se pregunta en los exámenes: si en un traceroute ves la IP pero no la MAC, es porque **cada router es un nuevo enlace con sus propias MACs**. Y si en un ejercicio te piden "las tramas del camino de C a F", lo que cambia entre filas es **siempre** la MAC y **nunca** la IP (salvo NAT, Clase 31).

(b) El objetivo: de "llegar al host" a "llegar al proceso"

```
──────────────────────────────────────────────────────────────────
CAPA 3 (Red):   "Este paquete tiene que llegar a la MÁQUINA
                 8.8.8.8."  El objetivo es un HOST.

CAPA 4 (Transporte): "Y una vez ahí, tiene que llegar al
                 PROCESO que escucha en el puerto 443 de esa
                 máquina, y no al que escucha en el 22 ni en el
                 53."  El objetivo es un PROCESO.
──────────────────────────────────────────────────────────────────

Por qué hace falta y por qué la capa 3 no alcanza:

Un servidor web en 8.8.8.8 tiene, típicamente, escuchando:
   :22    sshd
   :53    named (DNS)
   :80    httpd
   :443   httpd (TLS)
   :3306  mysqld
   :6379  redis

Las IP destino son TODAS 8.8.8.8. La IP no distingue.
La que distingue es el PUERTO, y por eso el puerto tiene que
ser un campo de la CABECERA DE CAPA 4.
```

La demostración clásica (y es un problema de examen): en un servidor, el *packet filter* de `iptables` / `nftables` puede hacer `DROP` por **puerto** porque la capa 3 le **pasa el paquete completo a la capa 4** en Linux (`netfilter` trabaja con cadenas por capa: `-p tcp --dport 443`). Ese es el motivo por el que un **firewall stateful implementa capas 3 y 4** (tabla de la Clase 1) y por el que la NAT necesita leer el puerto: **para poder multiplexar** varias conexiones internas detrás de una sola IP pública (Clase 31).

(c) Y un tercer cambio, más sutil: la granularidad de la direccionación

```
┌──────────────┬──────────────────┬──────────────────────────────────┐
│ Capa         │ ¿Qué identifica? │ Dónde está esa información       │
├──────────────┼──────────────────┼──────────────────────────────────┤
│ 4 Transporte │ una CONEXIÓN     │ 4-tupla (IP:port origen/destino) │
│ 3 Red        │ un HOST          │ IP destino (con prefijo /n)      │
│ 2 Enlace     │ una TARJETA      │ MAC destino                      │
│ 1 Física     │ un BIT           │ voltaje / fase / frecuencia      │
└──────────────┴──────────────────┴──────────────────────────────────┘

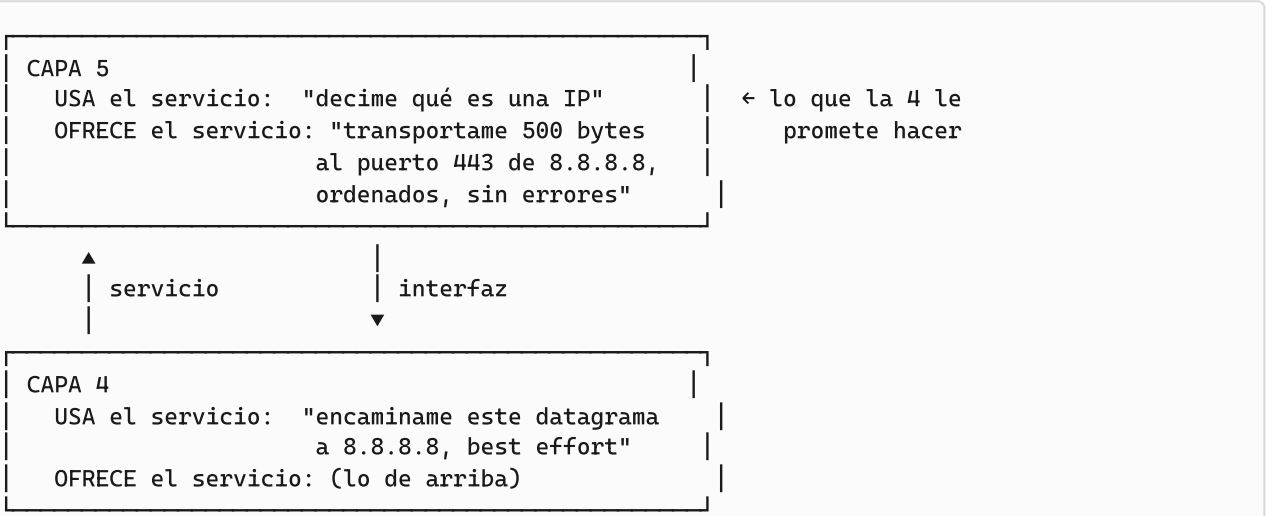
Y la granularidad va de uno a muchos:
 1 conexión TCP  →  1 paquete IP  →  1 trama  →  miles de bits
```

Un paquete IP puede contener varios segmentos TCP (eso es la segmentación de IP, distinta de la de TCP), y **una trama puede contener varios paquetes IP** si el medio lo permite. Por eso "un paquete" y "una trama" **no son sinónimos** y por eso los problemas de examen que mezclan PDU hay que mirarlos con cuidado.

1.5 Servicios e interfaces entre capas

Qué es un servicio

Cada capa **ofrece** un servicio a la de arriba y **usa** un servicio de la de abajo. "Servicio" no es un IND: es **una especificación de qué operaciones puede pedir la capa de arriba y qué garantías le da la capa de abajo**.



Los servicios de cada capa

Capa	**Qué le promete a la de arriba** (servicio)	**Qué espera de la de abajo** (usa)
7 Aplicación	Datos de aplicación, puffed a un proceso remoto	Un flujo de bytes confiable a un proceso remoto
6 Presentación	Datos en una **representación común**	Datos en la representación nativa
5 Sesión	Diálogo abierto, sincronizado, con recuperación	Un intercambio de datos ordenado
4 Transporte	Flujo **fiable y ordenado** de bytes entre procesos	Paquetes **no fiables** que pueden perderse o reordenar
3 Red	Paquetes **best effort** hacia un destino	Tramas confiables en un enlace
2 Enlace	Tramas **sin errores** entre dos vecinos	Bits crudos, que pueden llegar corruptos
1 Física	Bits, con la tasa de errores del medio	—

Por qué esto es la clave de la encapsulación

Porque **el contrato entre capas es lo que permite el intercambio**. Cuatro consecuencias concretas:

1. La capa de arriba no necesita saber cómo funciona la de abajo. El navegador HTTP no sabe que hay cobre debajo. La capa 4 (TCP) **no sabe si el camino tiene 3 o 25 saltos**, ni si los saltos son Ethernet o Wi-Fi, ni si hay pérdida. Y **por eso TCP funciona sobre cualquier cosa.**

2. Una capa puede implementarse de mil maneras. El servicio "encaminame paquetes a 10.0.0.7" lo pueden dar un router de 300 dólares con un chip o un backbone con routing fabric. La aplicación no se entera.

3. Los sniffers funcionan por capas. Un *packet capture* de capa 2 te da tramas. Uno de capa 3 te da datagramas. Uno de capa 4 te da segmentos. Y **`tcpdump`/Wireshark necesitás conocer el nivel para filtrar** (`-p` para capa 3). Si entendés los servicios, entendés por qué la herramienta tiene esa sintaxis.

4. Es lo que hace que se pueda cambiar una capa. Cambiar el medio (cobre → fibra) es cambiar la implementación de la capa 1 que **cumple el mismo servicio**. Cambiar el protocolo de transporte (TCP → QUIC) es cambiar la implementación de la capa 4. En los dos casos, **las capas de arriba no cambian**.

La relación entre servicio e interfaz

- La **interfaz** es **la frontera**: *dónde* se llama.
- El **servicio** es **el contrato**: *qué* se promete.

```
┌──────────────────────────────────────────┐
│  La interfaz es la PUERTA.               │
│  El servicio es lo que hay DEL OTRO LADO.  │
│                                          │
│  Si el servicio cambia (promete otra cosa), │
│  la implementación cambia.               │
│  Si la implementación cambia, la interfaz    │
│  puede quedar igual.                     │
└──────────────────────────────────────────┘
```

El ejemplo para explicar por qué el cambio de una capa no rompe las demás: si la capa 2 pasa de Ethernet a Wi-Fi, **la interfaz "encaminame este paquete a esta IP" sigue siendo exactamente la misma**. Wi-Fi cumple el mismo servicio de la capa 2 que cumple Ethernet: entregar tramas sin errores dentro de un alcance dado. Por eso un driver de IP no sabe si está sobre Ethernet o sobre Wi-Fi. Por eso tu laptop puede estar conectada a una red y no cambia una línea de código de tu navegador.

1.6 La capa de presentación, desarrollada

La capa 6 resuelve **el problema de la heterogeneidad de representación**. Lo desarrollamos con casos reales, porque es la capa que más se olvida y la que más puntos da.

(a) *Endianness* — el orden de los bytes

El problema. Un número de 32 bits con valor `0x12345678` ocupa 4 bytes. Esos 4 bytes se pueden ordenar de dos maneras:

```
BIG-ENDIAN ("network byte order", orden de red)
┌──────┬──────┬──────┬──────┐
│ 0x12 │ 0x34 │ 0x56 │ 0x78 │
└──────┴──────┴──────┴──────┘
 MSB                    LSB
 (peso 2³¹)             (peso 2⁰)
 se manda primero el de MAYOR peso

LITTLE-ENDIAN (orden de x86, ARM y casi todos los microcontroladores)
┌──────┬──────┬──────┬──────┐
│ 0x78 │ 0x56 │ 0x34 │ 0x12 │
└──────┴──────┴──────┴──────┘
 LSB                    MSB
 se manda primero el de MENOR peso

──────────────────────────────────────────────────────────────

El MISMO conjunto de 4 bytes significa:
   305 419 896   si es big-endian
2 018 915 346   si es little-endian

Si no me decís la endianness, el dato NO TIENE SIGNIFICADO.
──────────────────────────────────────────────────────────────
```

La regla de la red: todos los campos de más de 8 bits de **IP, TCP, UDP, ICMP** van en **network byte order (big-endian)**. Por eso en C existen `htons()`, `htonl()`, `ntohs()`, `ntohl()`.

```
Sin htons(), este código funciona en x86 y falla en big-endian:

    struct sockaddr_in sa;
    sa.sin_port = htons(80);      // htons converts to Network Order
    //  si pongo sa.sin_port = 80;  → funciona en little-endian
    //                              y FALLA en big-endian

Por eso la PI de la capa de presentación no es teórica:
es la razón por la que C tiene esas cuatro funciones.
```

(b) Formatos de caracteres y de archivos — el bug de UTF-8

El caso canónico del bug de UTF-8: tomemos la palabra **"mañana"** y veamos cómo se representa la letra Ñ:

```
La letra Ñ (U+00D1) en distintos esquemas:

ASCII      : no existe
ISO-8859-1 : 1 byte   0xD1
UTF-8      : 2 bytes  0xC3 0x91      (prefijo 110xxxxx + continuation)

Si el emisor manda ISO-8859-1 (0xD1) y el receptor asume UTF-8,
el receptor ve 0xD1 = un byte de inicio de un carácter de 2 bytes
y espera el siguiente, que es otra cosa. Resultado: "mojibake".

Si el emisor manda UTF-8 (0xC3 0x91) y el receptor asume ISO-8859-1,
el receptor ve dos caracteres: Ã (±) y Ñ. Resultado: "maÃ±ana".
```

Por qué UTF-8 ganó: es **backward-compatible**: un texto ASCII válido en UTF-8 es **idéntico** en ASCII. Eso permitió que durante años se usara UTF-8 en casi todo y no rompiera nada. La longitud es variable (1 a 4 bytes), y por eso "la cantidad de caracteres" **no** es "la cantidad de bytes".

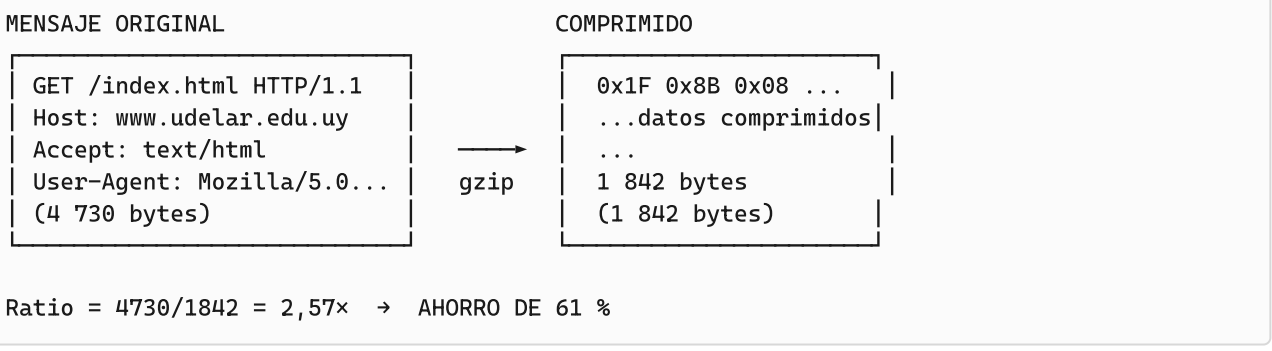
```
"hola"  en ASCII   = 4 bytes
"hola"  en UTF-8   = 4 bytes  (idéntico)
"ñ"     en ISO-8859-1 = 1 byte
"ñ"     en UTF-8   = 2 bytes
"😜"  en UTF-8   = 4 bytes  (¡y no existe en ASCII ni en ISO-8859-1!)
──────────────────────────────────────────────────────────────────

Un emoji ocupa 4 BYTES, no 4 caracteres. Es 1 carácter.
Si tu aplicación calcula "el string tiene 4 caracteres, mando 4 bytes",
manda la mitad de los bytes y rompe el mensaje.
```

Esto es un problema real de red, no de teoría: es la razón por la que el `Content-Length` de HTTP está en **bytes** y no en caracteres, y la razón por la que un cliente que anuncia `charset=iso-8859-1` y manda UTF-8 rompe la página.

(c) Compresión

Si el medio es caro (un enlace satelital de 300 Mbps que cuesta 4000 dólares al mes) o el tiempo es caro (un disco de 10 TB), **comprimir los datos antes de mandarlos** vale la pena.



El compromiso: la compresión **cuesta CPU** y **empeora la latencia** (hay que comprimir antes de mandar y descomprimir después de recibir). Para texto HTML (muy redundante) compensa muchísimo; para video ya comprimido (H.264, AV1) **empeora** la calidad sin ganar nada. Por eso `Content-Encoding: br` y `gzip` se aplican a texto, JSON y CSS, y **no** a video.

(d) Cifrado

La confidencialidad también es una función de "presentación": el dato tiene que ser **legible para el destinatario y no para nadie más**. En la práctica esto vive en **TLS** (`RFC 8446` , actualizado por `RFC 9846` en 2026), que es una biblioteca entre la aplicación y el socket.

Por qué en la capa de OSI y no en otra: porque el cifrado es **una transformación de la representación** del dato que debe hacerse de manera **transparente** a las capas 5 y 7. Si el cifrado fuera de capa 3 (como IPsec), cada host tendría que configurar la clave por ruta y no podría haber *NAT traversal*. Si fuera de capa 4, el *endpoint* de la aplicación tendría que saber del tema. En la capa 6, **la aplicación llama a una biblioteca y sigue escribiendo texto plano**.

La síntesis de la capa 6

```
┌──────────────────────────────────────────────────────────┐
│  CAPA 6 — PRESENTACIÓN                                   │
│                                                          │
│  Problema: los datos de un host pueden NO ser compatibles│
│            con los de otro, aunque los bits lleguen bien.│
│                                                          │
│  Cuatro funciones concretas:                             │
│    1. REPRESENTACIÓN: endianness, punto flotante, entero │
│    2. FORMATO:        codificación de caracteres, archivos│
│    3. COMPRESIÓN:     reducir lo que viaja               │
│    4. CIFRADO:        que solo el destinatario entienda  │
│                                                          │
│  En TCP/IP: no existe como capa. Vive en la APLICACIÓN,  │
│  y el cifrado en la biblioteca TLS.                      │
└──────────────────────────────────────────────────────────┘
```

La frase oficial del Examen 2006-12, textual:

> *"Busca la solución de problemas particulares, que hacen referencia a la semántica y sintaxis de lo enviado. Por ejemplo la resolución de diferentes representaciones entre máquinas (pto flotante), corresponde a esta capa."*

Análisis. La oficial dice **"semántica y sintaxis"**, que es más amplio que "los bytes": la capa 6 no solo traduce bytes, traduce **significados**. Y el ejemplo que da — el **punto flotante** — es exactamente *endianness* + **rango** + **precisión** + formato de la representación de un `float`. Los 4 bytes de un `float` de precisión simple: si el endianness difiere, el valor cambia; si el hardware no soporta ese `float`, el valor se convierte; si la longitud difiere (4 vs 8 bytes), el mensaje se rompe entero.

Lo que hay que agregar para el punto extra: que en TCP/IP estas funciones **no tienen capa propia**, y que la consecuencia práctica es que **cada protocolo de aplicación normaliza su representación** (JSON, protobuf, DER, X.509, JPEG, PNG), y que el cifrado se resuelve en TLS.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **El overhead de 58 bytes**: 20 de TCP + 20 de IP + 18 de Ethernet (14 + 4 de FCS). Es el número con el que hay que trabajar. - **La fórmula** `eficiencia = L / (L + overhead)` y el hecho de que el overhead relativo **se dispara con mensajes chicos**. - **MTU de Ethernet = 1500**; **MSS de TCP = 1460 = 1500 − 20 − 20**. - **Trama mínima de 64 bytes**, preámbulo de 7 + SFD de 1, e **IFG de 12 bytes entre tramas**. - **La IP no cambia en tránsito; la MAC cambia en cada router.** - **Que la capa de aplicación no ve la red.**

Cambió o se modificado:

- **IPv6 cambió el cálculo de la MTU.** En IPv4, el `Total Length` incluye **todas** las cabeceras. En IPv6, el campo **Payload Length excluye las cabeceras de extensión** (`RFC 8200` §3): los 40 bytes fijos de la cabecera IPv6 **no** están en el `Payload Length`. ⇒ **Cuando un problema de examen Fragmente un paquete IPv6, la cuenta de qué entra y qué no es distinta**, y es un error muy común tomarla de la fórmula IPv4.

- **El MTU mínimo absoluto de IPv6 es 1280 bytes** (`RFC 8200` §5, verificado textualmente). Por eso una conexión que "*funciona*" en IPv4 puede romperse en IPv6 si algún camino tiene MTU menor.
- **El MTU ya no es 1500 y ya no es una constante. Jumbo frames de 9000 bytes** son una **convención, no un estándar IEEE**: se usan sobre todo intra-datacenter y en almacenamiento, y **nunca** sobre Internet o túneles sin *clamp MSS*. Además, con **ECMP el PMTU es propiedad del camino, no del prefijo**, así que dos flujos al mismo destino pueden tener MTU distinto.
- **Cada overlay consume MTU.** MPLS, VXLAN (`RFC 7348`), IPsec y WireGuard restan bytes disponibles. Por eso el desajuste de MTU es **omnipresente** en redes cloud híbridas, y es el contenido de la Pregunta 2 del Examen 2010-12.
- **La capa 4 ya no es TCP ni UDP solamente.** QUIC (`RFC 9000`) **reemplaza a TCP en HTTP/3** y lleva su propio *header protection* y su propia recuperación. **El cifrado de la "capa de presentación" se mudó de sitio otra vez**: en QUIC, el *handshake* de TLS viaja en frames `CRYPTO`, no en un stream de bytes (`RFC 9001`). Un middlebox que solo entienda TCP/UDP plano no puede inspeccionarlo.
- **La fragmentación cambió por completo.** En IPv6 **el router nunca fragmenta**; fragmenta el origen. Y **DPLPMTUD (`RFC 8899`, 2022) reemplazó a la PMTUD clásica** porque ya no depende de ICMP: el transporte sondea con su propio paquete de datos. Es la respuesta canónica al *black hole*.
- **VLANs con etiqueta doble.** En carrier Ethernet y en EVPN-VXLAN el estándar real es **C-TAG (`0x8100`) + S-TAG (`0x88A8`)**, o sea **dos** cabeceras de capa 2. La encapsulación de capa 2 ya no es "una cabecera": es un stack.

> **Consecuencia para el examen:** los **números** cambian según la versión (MTU 1500 o 1280 mínimo, MSS 1460 o 1430), pero **el mecanismo** — encapsulation, overhead fijo, eficiencia `L/(L+H)`, IP global contra MAC local — **no cambió en 37 años**. Resolvé primero la aritmética con los números que dé el enunciado, y después agregá una línea de actualización.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Tamaño total y eficiencia para tres tamaños de mensaje

Datos. Sobre un enlace Ethernet: cabecera TCP de 20 B, cabecera IPv4 de 20 B, y trama Ethernet de 18 B de overhead (14 de cabecera + 4 de FCS). Se envían tres mensajes de aplicación de **20 B**, **800 B** y **9 000 B**.

a) ¿Cuánto viaja por el cable en cada caso? **b)** ¿Cuál es la eficiencia de cada uno? **c)** ¿Qué pasa con el mensaje de 9 000 bytes en una red con MTU de 1500?

Resolución.

Paso 1 — El overhead, una vez.

```
overhead = H_TCP + H_IP + H_ENLACE
         =   20   +  20  +     18
         =   58 bytes      (constante, independiente del mensaje)
```

Paso 2 — Los totales.

L_app	total = L + 58	bits	tramas (de 64 a
20 B	20 + 58 = 78 B	624 bits	1 trama (con pad)
800 B	800 + 58 = 858 B	6 864 bits	1 trama
9 000 B	9000 + 58 =9058 B	72 464 bits	fragmentado

Paso 3 — Las eficiencias.

```
Eficiencia = L_app / total

┌──────────┬──────────────────────┬──────────────┐
│ L_app    │ cuenta               │ eficiencia   │
├──────────┼──────────────────────┼──────────────┤
│   20 B   │  20 / 78             │ = 0,2564     │
│          │                      │ = 25,6 %     │
│          │                      │              │
│          │                      │ OJO: la      │
│          │                      │ Ethernet     │
│          │                      │ rellena con  │
│          │                      │ padding hasta│
│          │                      │ 64 bytes:    │
│          │                      │ 20/64 = 31,3%│
│          │                      │ (el mínimo   │
│          │                      │  de trama)   │
├──────────┼──────────────────────┼──────────────┤
│  800 B   │ 800 / 858            │ = 0,9324     │
│          │                      │ = 93,2 %     │
├──────────┼──────────────────────┼──────────────┤
│ 9 000 B  │ 9000 / 9058          │ = 0,9936     │
│          │                      │ = 99,4 %     │
│          │                      │ (¡con los    │
│          │                      │  fragments!) │
└──────────┴──────────────────────┴──────────────┘
```

Paso 4 — El mensaje de 9 000 bytes, parte c.

No pasa. El MTU de Ethernet son 1500 bytes de **payload de capa 2**, o sea del paquete IP completo:

```
MTU de Ethernet = 1500 bytes  ← esto mide el PAQUETE IP entero

Un paquete IP con 9000 B de datos TCP sería:
    9000 + 20 (TCP) + 20 (IP) = 9040 B de paquete IP
    9040 > 1500  ✗   NO CABE

Hay tres salidas:

(1) FRAGMENTAR (IPv4, con DF=0):
    Payload IP disponible = 1500 − 20 (IP) = 1480 B
    Fragmento 1: 20 (IP) + 1480 (TCP+datos)          = 1500 B
    Fragmento 2: 20 (IP) + 1480                       = 1500 B
    Fragmento 3: 20 (IP) + 1480                       = 1500 B
    Fragmento 4: 20 (IP) + 1480                       = 1500 B
    Fragmento 5: 20 (IP) + 1480                       = 1500 B
    Fragmento 6: 20 (IP) + 1480                       = 1500 B
    Fragmento 7: 20 (IP) + 1380                       = 1400 B
                                          TOTAL = 9020 B
    (9000 + 20 TCP + 7×20 IP = 9000 + 20 + 140 = 9160... ojo,
     calculemos bien abajo)

(2) DESCARTAR con ICMP (IPv4 con DF=1, o IPv6):
    ICMPv4 tipo 3 código 4 "Fragmentation Needed and DF was Set"
    El origen baja el MSS y vuelve a mandar. Sin DF=1, el
    router no puede avisar: se pierde el paquete.

(3) EL ORIGEN FRAGMENTA (IPv6, RFC 8200):
    En IPv6 el router NUNCA fragmenta. Fragmenta el origen.
```

La cuenta correcta de la fragmentación, con la aritmética a la vista:

```
Datos de aplicación            = 9 000 B
Cabecera TCP                   =    20 B   (un solo segmento)
──────────────────────────────────────────
Paquete IP a enviar            = 9 020 B

Capacidad por fragmento (MTU 1500):
   1500 − 20 (cabecera IP de cada fragmento) = 1480 B de payload

Payload a repartir: 9 000 + 20 = 9 020 B

Cantidad de fragmentos:
   ⌈9 020 / 1 480⌉ = ⌈6,0946⌉ = 7 fragmentos

Reparto:
   f1..f6 : 1480 B cada uno  = 8 880 B
   f7     : 9020 − 8880 = 140 B
   ──────────────────────────────
   total payload = 9 020 B  ✓

Overhead de fragmentar:
   7 × 20 B de cabecera IP = 140 B extra
   (comparado con los 20 B de un solo paquete)

Total que viaja en la red:
   7 tramas × (18 de Ethernet + payload_i)
     = f1..f6: 1480 + 20 + 18 = 1518 B cada uno  × 6 = 9 108 B
     = f7    :  140 + 20 + 18 =  178 B
     ──────────────────────────────────────────────
     TOTAL = 9 286 B

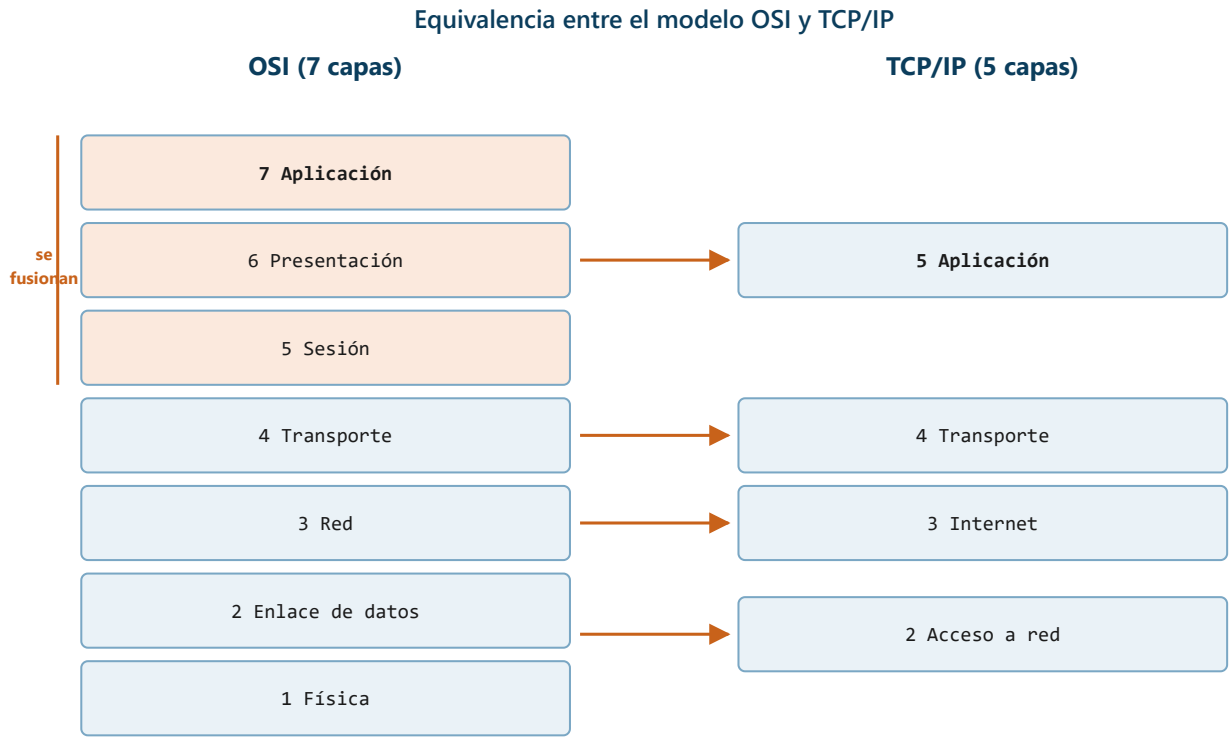
Eficiencia = 9 000 / 9 286 = 96,9 %
```

Interpretación. Fragmentar un mensaje de 9 000 bytes **agrega 140 B de overhead de IP** que no estaban en el cálculo original de 58 bytes. Y tiene un costo peor que el tamaño: **si se pierde UNO de los 7 fragmentos, se pierde el mensaje entero** (no hay protocolo de fiabilidad en IP), y hay que retransmitir 9 020 bytes. Es la razón por la que **DHCP, DNS y las respuestas HTTP grandes** están CCS штатно limitados al MTU. Por eso **DNS usa UDP con 512 bytes de payload** y por eso existe **TCP para respuestas grandes**.

⚠️ Errores típicos

- **Sumar el preámbulo (8 B) y el IFG (12 B) al tamaño de la trama.** El "tamaño de trama" en un problema de examen es `14 + payload + 4`. El preámbulo se cuenta aparte y el IFG va entre tramas.
- **Olvidar que cada fragmento lleva su propia cabecera IP de 20 B.** La fragmentación multiplica el overhead de capa 3.
- **Dividir mal: eficiencia = L/total, no = 1 − overhead/58.** El denominador es `L + 58`, que depende de `L`.
- **Confundir MTU con MSS.** MTU = 1500 (medido sobre el **paquete IP**). MSS = 1460 = 1500 − 20 (IP) − 20 (TCP) (medido sobre los **datos de aplicación**).

2.2 Ejercicio 2 — El camino completo: ¿quién lee qué?



En TCP/IP no existen Presentación ni Sesión:
el cifrado vive en la capa de Aplicación (TLS) y en la biblioteca.

Ojo: hay dos criterios de conteo. Tanenbaum cuenta 5 capas porque separa enlace y física; Kurose cuenta 4. La capa 3 es la misma.

Enunciado. Un cliente `192.168.1.25` (MAC `00:0C:29:AA:BB:CC`) quiere descargar una página de `203.0.113.9` (MAC del router de salida: `00:1A:2B:CC:DD:01`; MAC del switch: `00:AA:BB:00:00:01`). Hay un router `R1` en `192.168.1.1` (MAC `00:0C:29:FF:EE:01`) y un router `R2` en el otro lado. **Describí qué encapsula y qué desencapsula cada capa, y en qué momento exacto se invierte el proceso.**

Resolución — el camino, paso a paso.

```
═══ EN H1 (192.168.1.25) ═══════════════════════════════════

Cap 7  El navegador escribe:
        GET /index.html HTTP/1.1     Host: www.udelar.edu.uy
        (430 bytes)
        == write(socket_fd, buf, 430) ==
           ↓
Cap 4  TCP:
        · busca la conexión (4-tupla) o la crea (handshake)
        · segmenta: 430 B caben en un MSS → un segmento
        · NUMERA: este es el byte N del flujo
        · ANTECABE 20 B de cabecera TCP
          ┌──────────────────────────────────┐
          │TCP hdr (20)│ datos (430)         │ = 450 B  ← SEGMENTO
          └──────────────────────────────────┘
        == sendto() hacia 203.0.113.9:80 ==
           ↓
Cap 3  IP:
        · ¿203.0.113.9 está en mi subred (192.168.1.0/24)? NO
        · consulto la tabla: default gateway = 192.168.1.1
        · pongo IP origen = 192.168.1.25, IP destino = 203.0.113.9
        · TTL = 64, protocolo = 6 (TCP)
        · ANTECABE 20 B de cabecera IP
          ┌──────────────────────────────────────┐
          │IP hdr (20)│TCP hdr(20)│datos(430) │ = 470 B ← PAQUETE
          └──────────────────────────────────────┘
        == pasa a la capa 2 ==
           ↓
Cap 2  Ethernet:
        · ¿debo resolver una MAC? SÍ (el próximo salto es R1)
          → ARP request 192.168.1.25 → "¿192.168.1.1?"
          → ARP reply   R1 → "soy 00:0C:29:FF:EE:01"
        · armo la trama
          ┌──────────────────────────────────────────────┐
          │ETH hdr(18)│IP hdr(20)│TCP hdr(20)│datos(430)│ = 468 B
          │MAC:FF:EE:01 ← 00:0C:29:AA:BB:CC              │ ← TRAMA
          └──────────────────────────────────────────────┘
          (EtherType 0x0800 = IPv4)
        == transmito por el cable ==
           ↓
Cap 1  Codifico: 3744 bits de 0 y 1 en voltajes a 1 Gbps
        == (van por el cable) ==

═══════════ EN R1 ═══════════════════════════════════════

Cap 1  La NIC recibe voltajes → bits
        ↓
Cap 2  · verifico el FCS → coincide
        · MAC destino = 00:0C:29:FF:EE:01 = ¿la mía? SÍ
        · ENTREGO los 450 bytes a la capa 3
        · ⚠️ La capa 2 de R1 NO miró la IP. No sabe a dónde va.
          Solo sabe que la trama era para él.
        ↓
Cap 3  · leo la IP destino: 203.0.113.9
        · ¿es mía? NO
        · TTL = 64 − 1 = 63 → no es 0, sigue
        · recalculo el checksum de la cabecera IP
        · consulto la tabla de reenvío: 203.0.113.0/24 → eth1
        · entrego a la capa 2: "mandalo por eth1"
        ↓
Cap 2  · resuelvo (o tengo cacheada) la MAC de eth1
```

```
          · ARMO UNA TRAMA NUEVA: la vieja se descarta
            ┌──────────────────────────────────────────┐
            │ETH hdr(18)│IP hdr(20)│TCP hdr(20)│datos(430)│ = 468 B
            │MAC:AA:00:01 ← 00:0C:29:FF:EE:01          │
            └──────────────────────────────────────────┘
             ↑ MAC origen y destino CAMBIARON
             ↑ La cabecera IP es la MISMA, con TTL 63

═══════════ EN EL SERVIDOR (203.0.113.9) ═══════════════

Cap 1  bits → Cap 2: FCS ok, MAC destino = el mío → quito 18 B
        ↓
Cap 3  IP destino = 203.0.113.9 = MÍO → TTL −1 → quito 20 B
        ↓
Cap 4  TCP: checksum ok, seq esperado → anoto el byte,
        · quito 20 B → entrego 430 B a la aplicación
        ↓
Cap 7  El servidor web (puerto 80) lee 430 B y ve "GET /"

═══════ Y AHORA SE INVIERTE: la respuesta vuelve por el mismo camino ═══════

Cap 7  El servidor escribe la respuesta (5 200 B)
        ↓ ↓ ↓ (misma secuencia, sentido inverso)
Cap 4  TCP segmenta (4 segmentos), numera al revés
Cap 3  IP: 203.0.113.9 → 192.168.1.25
Cap 2  Eth:  → 00:0C:29:FF:EE:01   (¡ahora el origen es el servidor!)
Cap 1  bits
```

La respuesta a "¿en qué momento se invierte?"

Se invierte en el punto donde el paquete alcanza su destino, que es un **evento** y no un lugar:

```
El ÚNICO evento que marca el punto de inversión es:
la capa 3 del destino que RECONOCE su propia IP destino
en la cabecera del paquete.

Todo lo que está por debajo de ese punto va subiendo
(desencapsulando). Todo lo que está por arriba va bajando
(encapsulando).
```

Y hay un segundo punto de inversión, independiente del primero: cuando el paquete **sale de la capa 2 de H1 y entra en la capa 1**, se invierte la dirección **dentro del emisor**. Cada vez que un paquete entra a una capa, baja; cada vez que sale, sube. Un router hace subir a capa 3 y bajar a capa 3 por cada paquete.

La respuesta completa a la pregunta del enunciado:

Momento	Capa	Acción
El navegador llama `write()`	7→4	**Baja**: la 7 le pasa los datos a la 4
TCP encapsula	4→3	**Baja**: la 4 le pasa el segmento a la 3
IP encapsula	3→2	**Baja**: la 3 le pasa el paquete a la 2
ARP resuelve la MAC	2	**Consulta**: la 2 le pide a la 3 la dirección lógica de destino
Se arma y se manda la trama	2→1	**Baja**: la 2 le pasa los bits a la 1
La trama llega a H2	1→2	**¡SUBE!** La 1 le pasa los bits a la 2
El FCS da bien	2	Quita 18 B, entrega a la 3
La IP destino es la mía	3→4	**¡SUBE!** Quita 20 B, entrega a la 4
El segmento es de mi puerto	4→7	**¡SUBE!** Quita 20 B, entrega a la 7

⚠️ Errores típicos

- **Creer que la capa 3 del router "manda" el paquete.** El router **recibe la trama completa**, la entrega a la capa 3, la capa 3 lee la IP, decide, y **le dice a la capa 2 qué puerto usar**. La capa 2 arma la trama nueva. **El router no reescribe el paquete IP; la reescribe la capa 2.**
- **Creer que la IP cambia en tránsito.** No cambia, salvo por TTL, checksum y (fuera de NAT) los campos que NAT reescribe.
- **Pensar que ARP es un mensaje de capa 3 "normal".** ARP **no** es IP: no tiene cabecera IP, no tiene TTL, no se enruta. Va directo de capa 3 a capa 2 en un broadcast de enlace. Por eso **un router nunca reenvía ARP**.

2.3 Ejercicio 3 — ¿Qué pasa si el mensaje es de 40 bytes?

Datos. Un mensaje de aplicación de **40 bytes**. Cabeceras: TCP 20 B, IP 20 B, Ethernet 18 B.

a) ¿Cuánto viaja por el cable y cuál es la eficiencia? **b)** Con el mínimo de trama de 64 bytes de Ethernet, ¿cuál es la eficiencia real? **c)** ¿Cuántos bytes "útiles" quedan si además el protocolo de transporte es UDP (8 B de cabecera)?

Resolución.

a) Sobre el papel.

```
L_app    = 40 B
overhead = 20 (TCP) + 20 (IP) + 18 (Eth) = 58 B
──────────────────────────────────────────────
total    = 40 + 58 = 98 B

eficiencia = 40 / 98 = 0,4082 = 40,8 %
overhead   = 58 / 98 = 0,5918 = 59,2 %   ← MÁS DE LA MITAD
```

Interpretación: el 59 % del tráfico es andamiaje. Cada byte de dato útil arrastra 1,45 bytes de cabecera. Mandar 40 bytes de información cuesta 98 bytes de cable.

b) El mínimo de trama de 64 bytes.

La especificación IEEE 802.3 exige un mínimo de **64 bytes** (de preámbulo a FCS) para que el *slot time* de 512 bits permita detectar una colisión. Una trama más chica se rellena con *padding* después de los datos. **Cuidado con la cuenta**, porque es el error clásico de este ejercicio:

```
payload de la trama = 20 (IP) + 20 (TCP) + 40 (datos) = 80 bytes
longitud de la trama = 14 (cabecera) + 80 (payload) + 4 (FCS) = 98 bytes

98 > 64  →  NO hace falta padding.
```

El umbral exacto del padding, entonces:

```
Para que NO haya padding en Ethernet:
    payload de la trama ≥ 46 bytes
    46 = 64 (mínimo) − 14 (cabecera) − 4 (FCS)

Payload = cabeceras IP+TCP + datos = 40 + L
    → 40 + L ≥ 46  →  L ≥ 6 bytes de aplicación

Payload = cabeceras IP+UDP + datos = 28 + L
    → 28 + L ≥ 46  →  L ≥ 18 bytes de aplicación
```

Con 40 bytes de datos de aplicación no hay padding, y la eficiencia es la del apartado a): **40,8 %**. El padding es un problema **solo** de los mensajes de aplicación muy chicos, típicamente los paquetes de control (un `ACK` puro de TCP son 0 bytes de datos: `20 + 20 + 0 = 40` de payload, `14 + 40 + 4 = 58` de trama, y se rellenan 6 bytes de padding hasta 64). **Ahí la eficiencia es 0/64 = 0 %:** un Ethernet lleno de ACKs puros transmite bits que no son datos. Por eso las redes con mucho tráfico de ACK puro (redes de control) son especialmente ineficientes.

c) Con UDP en vez de TCP.

```
L_app   = 40 B
overhead = 8 (UDP) + 20 (IP) + 18 (Eth) = 46 B
──────────────────────────────────────────────
total   = 40 + 46 = 86 B

eficiencia = 40 / 86 = 0,4651 = 46,5 %
overhead   = 46 / 86 = 0,5349 = 53,5 %
```

La comparación, que es el punto del ejercicio:

	overhead	total	eficiencia
TCP + IP + Eth	58 B	98 B	40,8 %
UDP + IP + Eth	46 B	86 B	46,5 %
DIFERENCIA	−12 B	−12 B	+5,7 p.p.

Interpretación y por qué importa. Cambiar TCP por UDP **no cambia ni un byte** de los 40 de datos de aplicación, pero **reduce el tráfico total un 12,2 %** (`12/98`). Para una aplicación que manda 40 bytes por

paquete (un paquete de voz de 20 ms en un códec de G.711 con 8 bytes... o un mensaje de aplicación pequeño), ese 12 % es la diferencia entre que el enlace aguante o no.

Y ese es el argumento de VoIP (Clase 44): el códec de G.711 hace 8000 muestras de 8 bits por segundo = 64 000 bits/s = 8000 bytes/s. Si se pone en un paquete TCP de 160 bytes cada 20 ms:

```
160 bytes de audio → 178 con TCP → 186 con Eth  → eficiencia 86 %
overhead = 26 / 186 = 14 %
```

Con UDP de 160 bytes:

```
160 + 8 (UDP) + 20 (IP) + 18 (Eth) = 206 bytes
eficiencia = 160/206 = 77,7 %
overhead = 22,3 %
```

Y más importante que la eficiencia: **TCP retransmite**, y una retransmisión de voz a 20 ms de delay **no sirve para nada** (el destinatario ya pasó a la siguiente muestra). Y **TCP no puede porque quiere**: si el paquete se perdió, el destino ya tiene el hueco lleno de un paquete viejo. UDP manda y sigue. **Eso es lo que "no confiable" significa en la práctica: sin reintentos inútiles.** (Clase 44 lo desarrolla.)

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2009-02, Pregunta 2 (10 puntos) — forwarding vs. routing y flooding

> **Fuente:** Examen `sirc0902` — 2 de febrero de 2009, Pregunta 2 (10 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0902.pdf

Enunciado (texto del examen)

> **a)** En un contexto unicast, describa los procesos de forwarding y enrutamiento, y explique cómo se relacionan. **b)** En un contexto broadcast, describa un proceso de flooding, y explique cómo se puede transformar en flooding controlado.

Solución oficial (apartado a)

> El proceso de enrutamiento permite determinar el camino entre nodos de la red, típicamente mediante intercambio de información de topología y ejecución de un algoritmo en cada nodo para determinar el camino al resto de los nodos.
>
> El proceso de forwarding consiste en copiar cada paquete entrante a un nodo al puerto de salida adecuado para que siga su camino en la red (o entregarlo a un proceso local si el destinatario es el propio nodo). La tabla de forwarding, que asocia destinos con puertos (o interfaces) del nodo/router, se construye en base a la información de enrutamiento.

Solución oficial (apartado b)

> En un contexto broadcast, describa un proceso de flooding, y explique cómo se puede transformar en flooding controlado.
>
> ⚠️ **Nota sobre la solución oficial:** el apartado **b)** de la solución publicada **no tiene texto**. Solo está el enunciado. Hay que desarrollarlo. Lo marco como *(solución propia)* más abajo. También hay que notar que el enunciado del apartado **a)** de la solución oficial aparece con la letra "c)" en el PDF: es un error de maquetación de la cátedra.

Análisis de la solución oficial (apartado a)

Lo que está bien:

1. **La distinción de funciones es exacta**: "enrutamiento determina el camino", "forwarding copia al puerto de salida". Es la definición canónica.
2. **Dice que el forwarding puede entregar a un proceso local**: "*o entregarlo a un proceso local si el destinatario es el propio nodo*". Es un detalle que muchos alumnos olvidan y que muestra que el forwarding también tiene un caso "se queda acá".
3. **Dice que la tabla de forwarding SE CONSTRUYE a partir de la información de enrutamiento**: eso es **la relación entre los dos procesos**, que es lo que la pregunta pide explícitamente.

Lo que falta:

1. **No dice quién hace qué y cuándo.** Falta la clave temporal: el enrutamiento es **por período y por red** (o por topología); el forwarding es **por paquete y por interfaz**. Esa es la distinción operacional.
2. **No menciona el mecanismo de decisión**: el **longest prefix match** (*prefijo más largo que coincide*). Es lo que permite que las rutas se desambiguen y es el contenido de la Pregunta 1 del Examen 2011-03 (8 puntos).
3. **No habla del "next hop"**: la tabla de forwarding asocia destino con **interfaz**, pero en general también con **próximo salto** (*next hop*), que es lo que permite poner ARP para resolver la MAC del siguiente, no la del destino final. Ese detalle es lo que se pide en el Problema 2 del Examen 2011-03 y en el Problema 1 del Examen 2009-12.
4. **No menciona el plano de datos y el plano de control**, que es la forma moderna y más precisa de separar los dos procesos. La cátedra lo llama "plano de datos / plano de control" (Pregunta 4 del Examen 2008-07).
5. **No menciona IPv6 ni el *next header***, que es el mecanismo de extensión de IPv4 en IPv6 (RFC 8200): el *next header* va en la cabecera fija de IPv6, y el *protocol* en IPv4.

Mi respuesta completa (apartado a)

```
FORWARDING (reenvío) — plano de DATOS
  - Qué: para CADA PAQUETE que llega a una interfaz, decidir por
    qué puerto de salida sale, y copiarlo.
  - Granularidad: POR PAQUETE, POR INTERFAZ.
  - Información que usa: la TABLA DE REENVÍO (FIB), que es una
    estructura ACCELERADA en hardware (TCAM / Ternary CAM) dentro
    del chip de línea de tarjetas, y que se deriva de la RIB.
  - Decisión: LONGEST PREFIX MATCH sobre la IP DESTINO.
      "Busco en la tabla todos los prefijos que sean prefijo de la
       IP destino, y gana el MÁS LARGO."
  - Casos:
      · Si el destino es la propia dirección del router → entregar
        el payload a la capa de arriba (proceso local).
      · Si no hay ninguna entrada que coincida → descartar y, si
        corresponde, mandar ICMP Destination Unreachable.
      · Si hay coincidencia → reenviar por la interfaz indicada.
  - Determina también el NEXT HOP (dirección de capa 3 del
    siguiente router), que es a quien hay que resolver la MAC con
    ARP para armar la trama de salida.

ROUTING (enrutamiento) — plano de CONTROL
  - Qué: determinar el CAMINO (la tabla de reenvío) entre los
    nodos de la red.
  - Granularidad: POR PERÍODO y POR RED, no por paquete. Un
    router de borde recalcula BGP cada 30 s o cada 5 minutos; uno
    de OSPF recalcula cuando cambia la topología.
  - Mecanismo: intercambio de información de control con los
    vecinos (distance-vector, link-state, path-vector) y ejecución
    de un algoritmo (Bellman-Ford, Dijkstra, o el policy engine
    de BGP con sus atributos y sus políticas locales).
  - Produce: entradas en la RIB, que se seleccionan por preferencia
    (una tabla puede tener 10 rutas al mismo destino) y se
    traducen a la FIB.

_____________________________________________________________________


CÓMO SE RELACIONAN

     VECINOS ──(protocolo de enrutamiento)──► INFORMACIÓN
                                                  │
                                          ┌───────▼────────┐
                                          │  ALGORITMO     │
                                          │ (Bellman-Ford /  │
                                          │  Dijkstra / BGP) │
                                          └──────┬─────────┘
                                                 ▼
                                            RIB (candidatas)
                                                 │ selección
                                                 ▼
PAQUETE ──► FIB ──► consulta al NEXT HOP ──► AR
(de entrada)   (tabla de reenvío)     (si no está cacheado)
      ▲              │
      └──────────────┘
   el forwarding USA la tabla que el routing CONSTRUYÓ

- El routing NO mira paquetes. El forwarding NO habla con vecinos.
- El forwarding es RÁPIDO (decisión por hardware, ~100 ns) y por
  paquete. El routing es LENTO (CPU, segundos) y por topología.
```

```
- Si el forwarding no puede decidir (sin entrada), se puede pedir
  un reenvío a la capa de control (un "route request"), pero es
  el caso excepcional, no el normal.

- En IPv6 el mecanismo es el mismo, pero el campo se llama
  NEXT HEADER (RFC 8200), no "protocolo", y permite encadenar
  cabeceras de extensión en lugar de indicar un protocolo nuevo.

- La distinción unicast vs. broadcast no cambia las funciones:
  cambia la ENTRADA de la tabla que se busca. En unicast es una
  IP destino concreta; en broadcast es la dirección de broadcast
  del dominio, y el comportamiento de reenvío cambia (ver b).
```

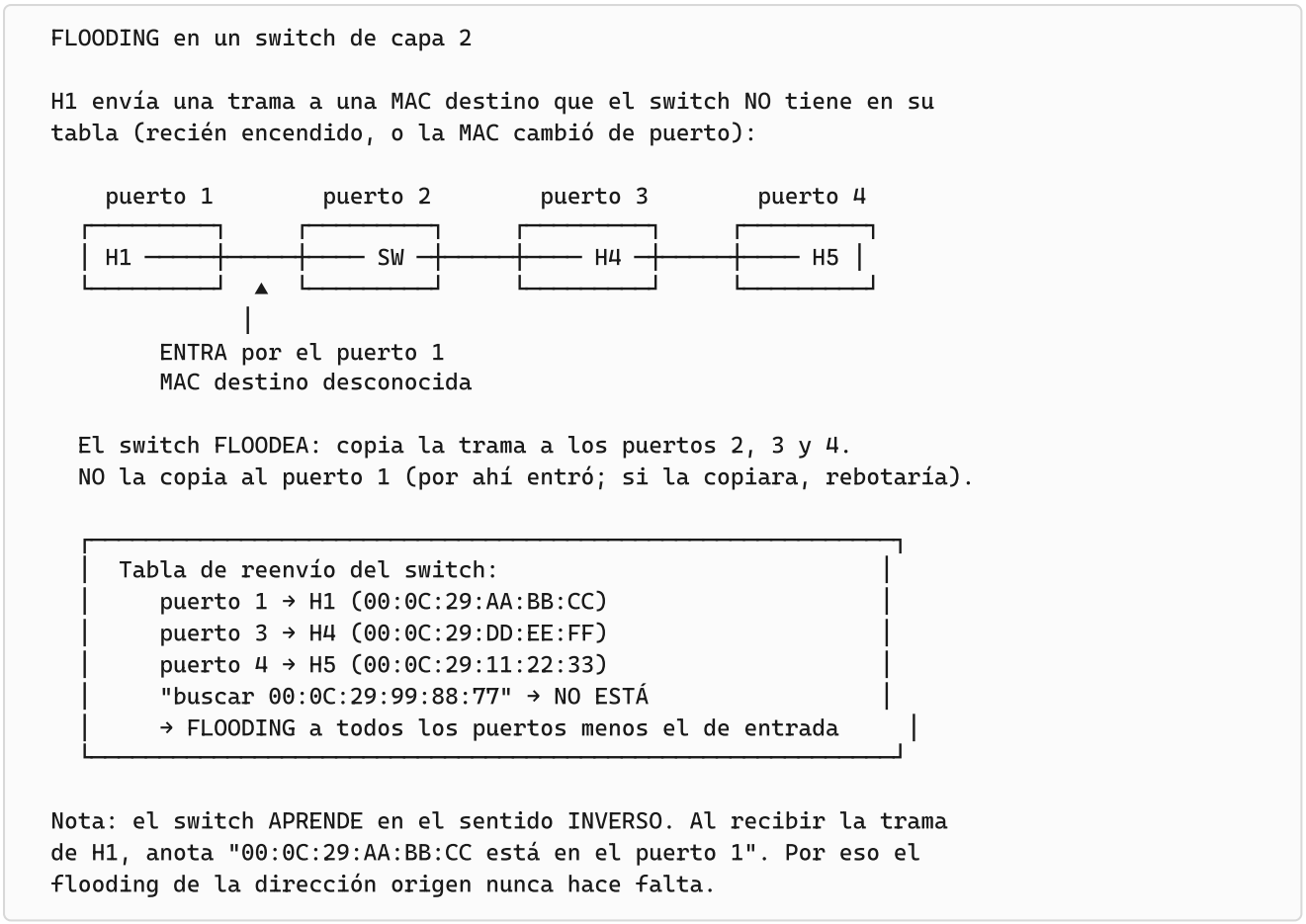
Los tres puntos extra que agregué y por qué:

Agregado	Por qué suma
Plano de datos vs. plano de control	Es la formulación moderna de la misma distinción, y es la Pregunta 4 del Examen 2008-07 (10 pts).
Longest prefix match	Es **el mecanismo concreto** de la decisión, y aparece en todos los problemas de tablas de ruteo.
Next hop ≠ destino final	Es lo que explica por qué en la tabla ARP del host está la MAC del **gateway**, no la del servidor.
Grano temporal (por paquete vs. por topología)	Da la idea de la distinción de una línea: "uno es instantáneo, el otro es una decisión lenta".
IPv6: next header	Actualización.

Apartado b) — flooding y flooding controlado *(solución propia)*

> El apartado b) no tiene solución publicada en el PDF oficial. Lo desarrollo completo.

Qué es flooding. Es el reenvío de un paquete **a todos los puertos de salida excepto aquel por el que entró**. Es lo que hace un switch cuando no sabe la MAC destino, y lo que hace un router con la dirección de broadcast de la LAN.



Por qué existe el flooding y por qué es necesario. Porque **no podés mantener una tabla de destinos completa si hay más destinos que puertos**. Un switch de 48 puertos puede tener learned miles de MACs; los destinos posibles son 2^{48}. La decisión es: **¿mantengo la tabla completa (memoria infinita) o flooding cuando no sé?** Se eligió flooding + aprendizaje. Y para **broadcast**, **no hay tabla posible**: el broadcast es por definición "todos", así que reenviar a todos los puertos **es** la semántica.

Cómo se transforma en flooding controlado. El flooding controlado es **flooding + restricciones**, y hay cinco restricciones distintas:

(1) No reenviar por el puerto de entrada. Es el caso base, y sin esto hay un **loop**. Es exactamente el problema de la Pregunta 2 del Examen 2007-03 (10 puntos): si un puente hace flooding a **todos** los puertos incluido el de entrada, cada trama vuelve, y los dos puentes se la copian el uno al otro **para siempre**. La solución oficial de ese examen es el **árbol de cubrimiento (spanning tree)**: los puentes se comunican, descubren la topología y **desactivan los puertos que crearían un bucle**, de modo que en la topología activa **solo hay un camino** entre cada par de LANs.

(2) Controlar el alcance con el TTL. Es lo que hace un router: si reenvía un broadcast, **decrementa el TTL**, y cuando llega a 0 lo descarta. Eso acota el flooding a un número de saltos y evita que un router que no pertenece al dominio reenvíe. El comportamiento por defecto de los routers es **NO reenviar broadcasts** (se llama *split horizon* en el contexto de VLSM... no: se llama **no forwarding de broadcasts**, o "límite de TTL"). La RFC 1812 lo define: un router **no debe** reenviar datagramas con dirección de broadcast o multicast, salvo casos explícitos.

(3) Limitar por dominio: separar los dominios de broadcast. Cada VLAN es un dominio de broadcast independiente, y los routers **no reenvían** broadcasts entre VLANs. Con un router-on-a-stick o con un

switch L3, el flooding queda contenido dentro de cada VLAN. Esta es la razón de ser de las **VLAN** (Pregunta 3 del Examen 2009-03, 6 puntos).

(4) Filtrar por fuente conocida. Un switch **no reenvía un paquete de una MAC que acaba de aprender por el mismo puerto del que llegó**: eso es *ingress filtering* y corta el primer tipo de loop. Y un router con *uRPF* (*unicast reverse path forwarding*, RFC 3704) descarta cualquier paquete cuya dirección **origen** no sea alcanzable por el mismo puerto por el que llegó, lo que corta el **spoofing** y muchos loops de reenvío.

(5) Limitar el ritmo. Cuando el flooding es deliberado (link-state de OSPF, ARP en un dominio grande), hay que acotar el daño de una tormenta de broadcast con *rate limiting* por puerto, o descartando tramas duplicadas dentro de un intervalo corto.

El diagrama del flooding controlado por TTL:

```
H1 ─── S1 ─── R1 ─── R2 ─── R3 ─── S2 ─── H2
       │         │       │       │       │        │
       └─────────┴───────┴───────┴───────┘

       dominio de broadcast de H1 (TTL = 64)
       ──────────────────────────────────

       R1 reenvía: TTL 64 → 63
       R2 reenvía: TTL 63 → 62
       R3 reenvía: TTL 62 → 61
       ──────────────────────────────────────────

       H2, que está a 4 saltos, NO lo recibe:
       su dominio de broadcast es otro (el de S2).

→ El flooding queda acotado por SALTOS (TTL) y por
  SEGMENTACIÓN en dominios de broadcast (routers y VLANs).
```

Las dos comparaciones que hay que saber para el examen:

UNICAST	BROADCAST
1 destino	todos los destinos del dominio
Tabla: sí, se puede mantener	Tabla: NO PUEDE existir (2⁴⁸ destinos) → flooding por definición
Unknown unicast: ARP en el host, o flooding si no hay entrada	Unknown es el caso NORMAL: toda trama de ARP request es broadcast; la primera respuesta gana (el host descarta la segunda) → "first answer wins"
Se controla con tabla + ARP	Se controla con: TTL, routers que no reenvían, VLANs, spanning tree

El caso especial de ARP que demuestra por qué el flooding es necesario y cómo se controla:

```
ARP request  "¿192.168.1.50 está?"
    → BROADCAST obligatorio (no sabemos a quién preguntarle)
    → llega a TODOS los hosts del dominio  (esto es flooding)

ARP reply  "sí, soy yo: 00:0C:29:AA:BB:50"
    → UNICAST al que preguntó
    → Y además: se cachea en todos los que escucharon
      (por eso el gratuitous ARP es una fuente clásica de
       ARP spoofing / cache poisoning)

CONTROL DEL ALCANCE:
    · si la pregunta es de OTRA subred (target fuera del /24),
      el host NO hace ARP: manda el paquete al gateway, porque
      la MAC no tiene sentido fuera del enlace.
    · un router NUNCA reenvía un ARP request de otro dominio.
    · un bridge solo reenvía dentro de su dominio de broadcast.
```

⚠️ Errores típicos del apartado b)

- **Confundir flooding con "reenviar a todos los puertos incluido el de entrada".** Eso es lo que genera la tormenta de broadcast. El flooding **excluye** el puerto de entrada.
- **Decir que el flooding se controla "con la tabla".** No: la tabla **no** sirve para broadcast, porque el broadcast es todos los destinos. Se controla con TTL, con routers, con VLANs y con spanning tree.
- **Olvidar que el switch aprende del sentido inverso.** El flooding nunca es necesario para una dirección origen recién aprendida.
- **Confundir "dominio de broadcast" con "dominio de colisión".** En una red conmutada hay **muchos dominios de colisión (uno por puerto) y UNO de broadcast**. En una red con hubs, hay **uno de cada uno**. Es la Pregunta 3 del Examen 2010-12.

3.2 Referencia cruzada — la misma pregunta, en otro examen

> **Examen `sirc1103` — 21 de febrero de 2011, Pregunta 1 (8 puntos):** *"Describa claramente las dos funciones clave de la capa de red y explique cómo interactúan entre ellas."*

Es la misma pregunta del apartado a), pero más detallada y con 8 puntos. La solución oficial de 2011-03 es la mejor que hay del tema, y vale la pena citarla entera porque muestra **cómo responde la cátedra**:

> Reenvío (forwarding): mover paquetes entre puertos de entrada y salida del router.
>
> Enrutamiento (routing): determinar la ruta de los paquetes desde origen a destino. Para ello existen los algoritmos de enrutamiento.
>
> Interacción: Cada router intercambia con sus pares, mediante al menos un protocolo de enrutamiento, información que se utiliza, en al menos un algoritmo de enrutamiento, para poblar sus tablas de forwarding. Dichas tablas, en su formato más básico, contienen, para un valor de dirección contenido en la cabecera de los paquetes que llegan al router, el enlace de salida. De esta forma, para cada paquete que arriba a un router por cualquiera de sus interfaces, el router examina la tabla

> de forwarding (en el caso de IP a partir de la dirección IP destino contenida en el encabezado IP) y a partir del **"longest match"** determina por qué interfaz debe ser enviado (además de determinar cuál es el **"next hop"**, lo que determina en el caso más genérico, la dirección destino de capa de enlace que deberá contener la trama que lo lleve a su próximo salto).

Por qué esta solución es excelente y qué copiar de ella:

1. **Separa los dos procesos con una frase cada uno** y sigue.
2. **Nombra el mecanismo de la interacción**: protocolo de enrutamiento → algoritmo → tabla de forwarding.
3. **Dice que la tabla, "en su formato más básico", contiene el enlace de salida** — y **agrega que además contiene el next hop**, que es el refinamiento que hace que la tabla sirva para armar la trama correcta.
4. **Nombra el "longest match"** explícitamente.
5. **Dice "por cualquiera de sus interfaces"**, que es el detalle de que la tabla es **una sola**, no una por interfaz.

Lo que yo agregaría (ya está en mi respuesta del apartado a): **plano de datos vs. plano de control**, **granularidad temporal**, **RIB → FIB**, y el **next header de IPv6**.

PARTE 4 — CIERRE

📌 Resumen

1. **Encapsular** es anteponer la cabecera de la capa al bajar por la pila. **Desencapsular** es quitarla al subir. **Cada capa quita exactamente los bytes que antepuso.**
2. **El byte viajando:** 500 B de aplicación → **520** (con TCP) → **540** (con IP) → **558** en el medio. Overhead: `20 + 20 + 18 = 58 B`.
3. **Eficiencia =** `L / (L + 58)`. A 500 B es 89,6 %; a 50 B es **46,3 %**; a 40 B es **40,8 %**. El overhead es constante y el mensaje no, así que **el overhead relativo se dispara con mensajes chicos**.
4. **Encapsular por cuatro razones:** control local, **multiplexación** (el router lee el destino sin abrir el sobre), que cada capa hable su idioma, e **independencia** (cambiar una capa sin arrastrar a las demás).
5. **Lo que además cambia:** el **direccionamiento** (IP es global y jerárquica, MAC es local y plana; la IP no cambia en tránsito, la MAC cambia en cada router) y el **objetivo** (capa 3 lleva al **host**, capa 4 lleva al **proceso**).
6. **Servicio** = el contrato entre capas (qué promete la de abajo); **interfaz** = dónde se llama. Gracias a eso se puede cambiar una capa sin tocar las demás.
7. **La aplicación NUNCA ve la red.** Le pasa bytes a la capa 4 y recibe bytes. Todo lo demás lo decide la pila.
8. **Presentación** resuelve la heterogeneidad de representación: *endianness* (big-endian en red), codificación (UTF-8), formato, compresión y cifrado (TLS). En TCP/IP no existe como capa.
9. **Forwarding** = por paquete, por interfaz, con *longest prefix match*. **Routing** = por topología, con un algoritmo. El segundo construye la tabla del primero.
10. **Broadcast**: no hay tabla posible, así que **flooding** por definición. **Flooding controlado** = excluir el puerto de entrada (spanning tree), acotar con **TTL**, separar con **routers y VLANs**, y filtrar por origen (*ingress filtering*).

✅ Autoevaluación

1. Un mensaje de 1200 bytes de aplicación. ¿Cuánto viaja por el cable y cuál es la eficiencia?

▶ Respuesta

2. ¿Por qué la cabecera va al principio y no al final del mensaje?

▶ Respuesta

3. Un router reenvía un paquete. ¿Cambia la cabecera IP? ¿Por qué?

▶ Respuesta

4. En el modelo TCP/IP, ¿cuántas PDU hay? ¿Son las mismas que en OSI?

▶ Respuesta

5. ¿Por qué un firewall stateful necesita las capas 3 y 4?

▶ Respuesta

6. Explicame por qué la capa de aplicación "no ve la red" con un ejemplo concreto.

▶ Respuesta

7. ¿Qué cambia entre un mensaje de 46 bytes y uno de 60 bytes, ambos con TCP sobre IPv4?

▶ Respuesta

8. ¿Qué es flooding controlado? Enumerá las cinco restricciones.

▶ Respuesta

🔗 Referencias

Exámenes - Examen `sirc0902` (2-feb-2009) — P2 (10 pts): forwarding vs. routing y flooding controlado · P4 (9 pts): datagramas vs. circuitos virtuales → **Clase 4** - Examen `sirc1103` (21-feb-2011) — P1 (8 pts): las dos funciones de la capa de red (la mejor solución oficial del tema) · Problema 2 (30 pts): intercambio de tramas con ARP - Examen `sirc0612` (26-dic-2006) — P1 (5 pts): modelo OSI y Presentación - Examen `sirc0703` (26-feb-2007) — P2 (10 pts): puente con flooding y spanning tree · P3 (10 pts): RARP - Examen `sirc0707` (31-jul-2007) — Problema 2b (25 pts): dos hosts que comparten una IP y responden ARP - Examen `sirc0912` (18-dic-2009) — Problema 1 (30 pts): tablas de ruteo y secuencia de tramas con ARP vacío - Examen `sirc1003` (20-mar-2010) — P1 (8 pts): ARP entre dos hosts con MAC e IP

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., **cap. 1 §1.4** "El modelo de capas" (pág. ~20 a ~26): encapsulación, PDU y servicios entre capas - Tanenbaum & Wetherall, **cap. 1 §1.6** "El modelo TCP/IP" — la comparación de las dos columnas - Tanenbaum & Wetherall, **cap. 2** — tramas de Ethernet, MTU 1500 y tamaño mínimo de 64 bytes (pág. ~45 a ~60) - Kurose & Ross, *Computer Networking: A Top-Down Approach*, cap. 1 §1.7-1.8 — el modelo de 4 capas y la encapsulación con el diagrama de "un byte en la red"

RFCs - `RFC 8200` — *Internet Protocol, Version 6* (2017, *Internet Standard*): el campo **Next Header** en vez de "protocolo" - `RFC 9293` — *TCP* (2022, *Internet Standard*): los 20 bytes de cabecera y el campo *payload offset* que permite encontrar la cabecera IP sin recorrer la trama - `RFC 768` — *UDP* (1980): **8** bytes de cabecera, la mitad de TCP - `RFC 826` — *ARP* (1982): por qué no tiene cabecera IP y por eso no se enruta - `RFC 1812` — *Requirements for IP Routers* (1995): el comportamiento por defecto de **no reenviar broadcasts**, y el conjunto de restricciones de *ingress filtering* que acota el flooding - `RFC 3704` — *Unicast Reverse Path Forwarding* (2002): el filtro por dirección origen que corta loops y spoofing - `RFC 7541` — *HPACK: Header Compression for HTTP/2* (2015): cómo la capa de presentación (compresión de cabeceras) sigue teniendo su función en TCP/IP

Clase siguiente: 4 — Conmutación: circuitos, paquetes y mensajes